

Elevator Controller Project Report

1. Project Description

This project implements a Finite State Machine (FSM) based Elevator Controller using Verilog HDL. The controller compares the requested floor with the current floor and moves the elevator upward or downward. After reaching the requested floor, it opens the door for one clock cycle and returns to the IDLE state.

2. Objectives

- Design an FSM-based elevator controller.
- Implement the design in Verilog HDL.
- Verify the functionality using a testbench.
- Generate RTL schematic using Vivado.

3. FSM Description

State	Description
IDLE	Waits for a floor request.
MOVE_UP	Moves the elevator upward.
MOVE_DOWN	Moves the elevator downward.
OPEN_DOOR	Stops the motor, opens the door, then returns to IDLE.

4. Design Code Explanation

The elevator_controller module contains four FSM states: IDLE, MOVE_UP, MOVE_DOWN, and OPEN_DOOR. On reset, the elevator starts at floor 0. If request_floor is greater than current_floor, the controller enters MOVE_UP. If lower, it enters MOVE_DOWN. When the destination is reached, door_open is asserted for one clock cycle.

5. Design Code

```
`timescale 1ns / 1ps
```

```
module
```

```
elevator_controller(clk,reset,request_floor,current_floor,motor_up,motor_down,door  
_open);
```

```
    input clk;
```

```
    input reset;
```

```
    input [2:0] request_floor;
```

```

output reg [2:0] current_floor;
output reg motor_up;
output reg motor_down;
output reg door_open;

//-----
// State Encoding
//-----

parameter IDLE    = 2'b00;
parameter MOVE_UP  = 2'b01;
parameter MOVE_DOWN = 2'b10;
parameter OPEN_DOOR = 2'b11;

reg [1:0] state;

//-----
// FSM
//-----

always @(posedge clk or posedge reset)
begin

    if(reset)
    begin
        state <= IDLE;
        current_floor <= 2'd0;

        motor_up <= 0;
        motor_down <= 0;
        door_open <= 0;
    end

    else
    begin

        case(state)

            //-----
            // IDLE
            //-----

            IDLE:
            begin

                motor_up <= 0;
                motor_down <= 0;

```

```

    door_open <= 0;

    if(request_floor > current_floor)
        state <= MOVE_UP;

    else if(request_floor < current_floor)
        state <= MOVE_DOWN;

end

//-----
// MOVE UP
//-----

MOVE_UP:
begin

    motor_up <= 1;
    motor_down <= 0;
    door_open <= 0;

    current_floor <= current_floor + 1;

    if(current_floor + 1 == request_floor)
        state <= OPEN_DOOR;

end

//-----
// MOVE DOWN
//-----

MOVE_DOWN:
begin

    motor_up <= 0;
    motor_down <= 1;
    door_open <= 0;

    current_floor <= current_floor - 1;

    if(current_floor - 1 == request_floor)
        state <= OPEN_DOOR;

end

//-----
// OPEN DOOR
//-----

```

```

OPEN_DOOR:
begin

    motor_up <= 0;
    motor_down <= 0;
    door_open <= 1;

    state <= IDLE;

end

endcase

end

end

endmodule

```

6. Testbench Explanation

The testbench generates a 10 ns clock, applies reset, requests floors 3, 1, 0, and 4, and monitors the outputs using the \$monitor system task.

7. Testbench Code

```

`timescale 1ns / 1ps

module tb_elevator_controller;

//-----
// Inputs
//-----

reg clk;

reg reset;

```

```
reg [2:0] request_floor;
```

```
//-----
```

```
// Outputs
```

```
//-----
```

```
wire [2:0] current_floor;
```

```
wire motor_up;
```

```
wire motor_down;
```

```
wire door_open;
```

```
//-----
```

```
// Instantiate DUT
```

```
//-----
```

```
elevator_controller
```

```
DUT(clk,reset,request_floor,current_floor,motor_up,motor_down,door_open);
```

```
//-----
```

```
// Clock Generation (10 ns Period)
```

```
//-----
```

```
always #5 clk = ~clk;
```

```
//-----
```

```
// Test Sequence
```

```
//-----
```

```
initial
```

```
begin
```

```
    clk = 0;
```

```
    reset = 1;
```

```
    request_floor = 2'd0;
```

```
    // Apply Reset
```

```
    #20;
```

```
    reset = 0;
```

```
//-----
```

```
// Move to Floor 3
```

```
//-----
```

```
    #20;
```

```
    request_floor = 2'd3;
```

```
    #60;
```

```
//-----
```

```
// Move to Floor 1
```

```
//-----
```

```
request_floor = 2'd1;
```

```
#50;
```

```
//-----
```

```
// Move to Floor 0
```

```
//-----
```

```
request_floor = 2'd0;
```

```
#40;
```

```
request_floor = 3'd4;
```

```
#100;
```

```
//-----
```

```
// Finish Simulation
```

```
//-----
```

```
#20;
```

```
$finish;
```

```
end
```

```
//-----
```

```
// Monitor
```

```
//-----
```

```
initial
```

```
begin
```

```
    $monitor("Time=%0t | Req=%0d | Current=%0d | UP=%b | DOWN=%b | DOOR=%b",
```

```
        $time,
```

```
        request_floor,
```

```
        current_floor,
```

```
        motor_up,
```

```
        motor_down,
```

```
        door_open);
```

```
end
```

```
endmodule
```

8. ModelSim Run Commands

```
vlib work
```

```
vlog elevator_controller.v
```

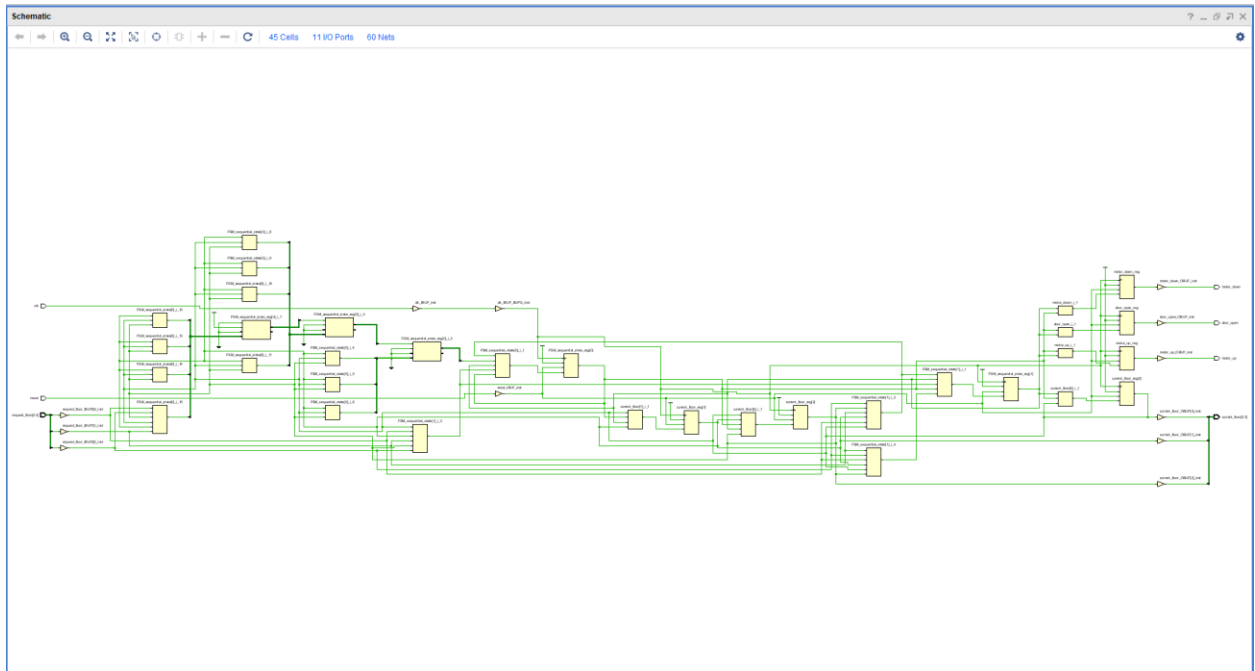
```
vlog tb_elevator_controller.v
```

```
vsim work.tb_elevator_controller
```

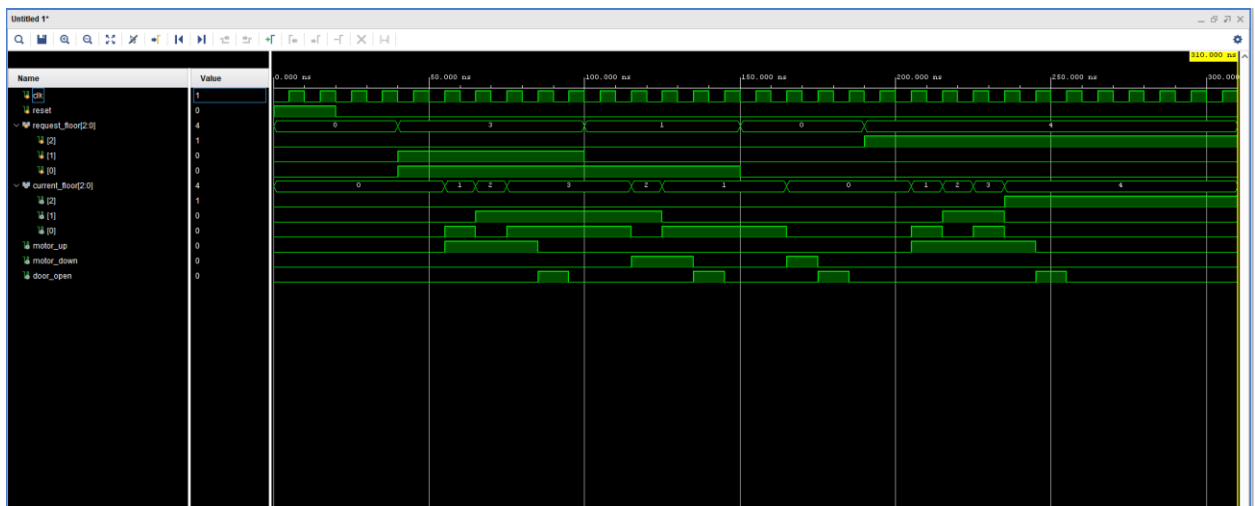
```
add wave -r *
```

```
run -all
```


9. Simulation Waveform



10. RTL Schematic



11. Applications

- Residential elevators
- Commercial buildings
- Industrial lifts
- Smart buildings
- Automated parking systems

12. Conclusion

The project successfully demonstrates an FSM-based elevator controller using Verilog HDL. The simulation and RTL schematic verify correct operation and synthesizability.